A2. Three Tiers of Gym Membership

Category B Assignment - Problem 2 (Basic to Intermediate)

Approach

EliteMember extends PremiumMember; GroupClassMember extends GymMember independently. Classification uses instanceof, checking the deepest subtype first. Total sessions are summed polymorphically without inspecting each type.

Java Solution

```
class GymMember {
    protected final String memberId;
    protected final int monthlyFee;
    protected int sessionsAttended;
    public GymMember(String memberId, int monthlyFee) {
        if (memberId == null || memberId.trim().length() < 4)
            throw new IllegalArgumentException("Invalid member ID");
        if (monthlyFee <= 0) throw new IllegalArgumentException("Fee must be positive");
        this.memberId = memberId; this.monthlyFee = monthlyFee;
    }
    public void attendSession() { sessionsAttended++; }
    public int getSessionsAttended() { return sessionsAttended; }
    public String displayInfo() { return "Standard Member | Sessions: " + sessionsAttended; }
}
class PremiumMember extends GymMember {
    protected final String trainerName;
    public PremiumMember(String id, int fee, String trainerName) {
        super(id, fee); this.trainerName = trainerName;
    }
    @Override public String displayInfo() {
        return "Premium Member | Trainer: " + trainerName + " | Sessions: " + sessionsAttended;
    }
}
class EliteMember extends PremiumMember {
    private final String lockerNumber;
    public EliteMember(String id, int fee, String trainer, String locker) {
        super(id, fee, trainer); lockerNumber = locker;
    }
    @Override public String displayInfo() {
        return "Elite Member | Trainer: " + trainerName + " | Locker: " + lockerNumber + " | Sessions: " + sessionsAttended;
    }
}
class GroupClassMember extends GymMember {
    private final String className;
    public GroupClassMember(String id, int fee, String className) {
        super(id, fee); this.className = className;
    }
    @Override public String displayInfo() {
        return "Group Class Member | Class: " + className + " | Sessions: " + sessionsAttended;
    }
}
class MembershipReport {
    static String classifyGeneration(GymMember member) {
        if (member instanceof EliteMember) return "Multilevel descendant (3 generations deep)";
        if (member instanceof GroupClassMember) return "Hierarchical sibling (independent branch)";
        if (member instanceof PremiumMember) return "Direct subclass of GymMember";
        return "Base class";
    }
    static int getTotalSessionsAttended(GymMember[] members) {
        int total = 0;
        for (GymMember member : members) total += member.getSessionsAttended();
        return total;
    }
}
```

Sample Output

```
classifyGeneration(elite) -> Multilevel descendant (3 generations deep)
classifyGeneration(groupClass) -> Hierarchical sibling (independent branch)
Total sessions (3 + 2 + 4) -> 9
```